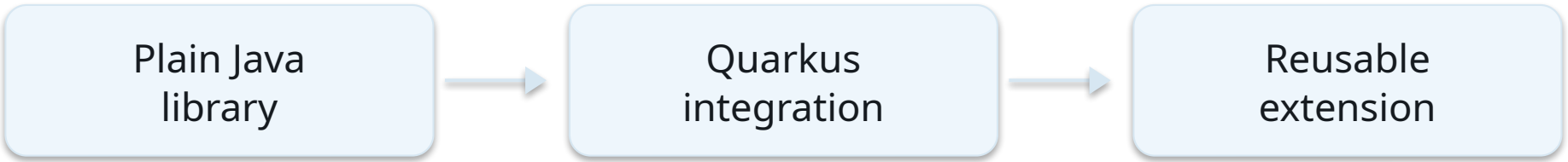
JavaCro'26

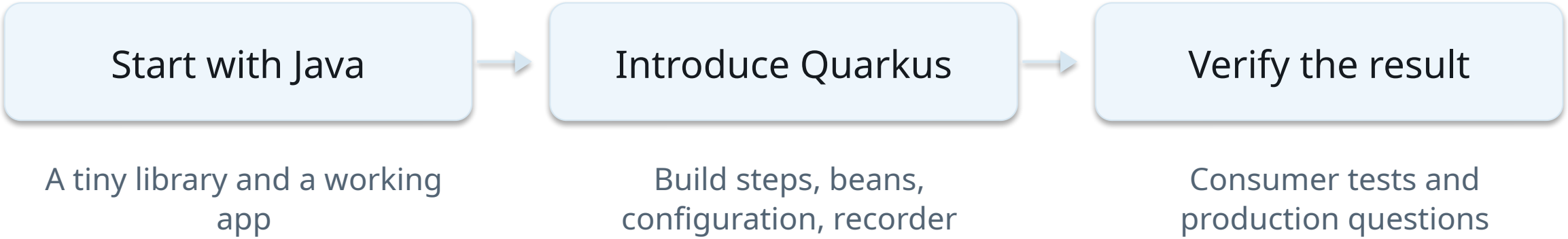
Building Quarkus Extensions

A Practical Guide for Java Developers



Not an extension API tour

We will make a working integration reusable, one mechanism at a time.



The goal: know when to build an extension and how to start.

// TODO

We will alternate short explanations with live code and tests.

Problem + baseline

`demo-00`

Architecture

`demo-01`

Build the extension

`demo-02 / 03 / 04`

Test + harden

`demo-05`

Takeaways + Q&A

`demo-final`

The code checkpoints are real Git tags, not a simulated sequence.

About me



Peter Velychko

- Software Developer
- Java, Kotlin, Scala

- LinkedIn: https://www.linkedin.com/in/petervelychko/

Quarkus in 60 seconds

A Java framework designed for cloud-native applications and a productive development loop.

Java, familiar APIs

REST, CDI, configuration and established Java libraries.

Developer experience

Dev mode, live reload, tests and extension-based integrations.

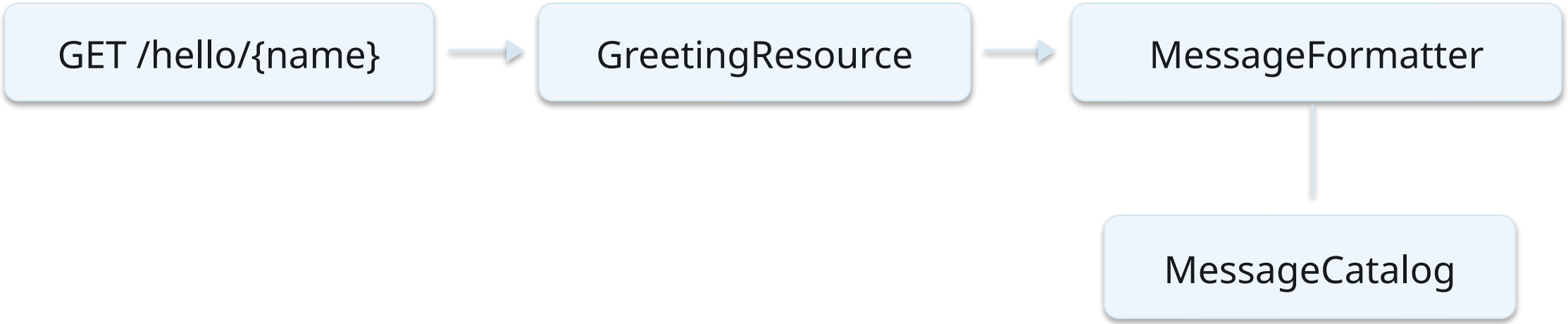
Build-time preparation

Index and process metadata during augmentation to reduce startup work.

Quarkus can run on the JVM or as a native executable.

The application is deliberately small

A REST endpoint makes the effect of the integration visible immediately.




```
curl http://localhost:8080/hello/Ada
Hello Ada
```

A tiny library keeps the focus on the integration model.

Baseline: ordinary CDI works

Before inventing an extension, prove the library and the application already work.

DEMO 00 **Manual integration inside the application**

```
@ApplicationScoped
public class MessageFormatterProducer {
    @Produces
    @Singleton
    MessageFormatter formatter() {
        return new MessageFormatter(...);
    }
}
```

What already works?

- Plain Java dependency
- CDI producer
- REST endpoint
- HTTP test

If this solves the problem, stop here.

Not every integration needs an extension

Choose the smallest solution that covers the real requirements.

Java dependency	Use the library directly	No Quarkus-specific work
CDI producer	Reuse beans and producers	Runtime wiring is enough
Quarkus extension	Integrate with augmentation	Build-time processing or deep framework support

An extension is an architectural choice, not a default dependency wrapper.

What problems can an extension solve?

The value is Quarkus-native integration, not just a new artifact.

Build-time work

Index, inspect, validate and prepare before application startup.

Developer experience

Register beans, expose config and reduce repeated app setup.

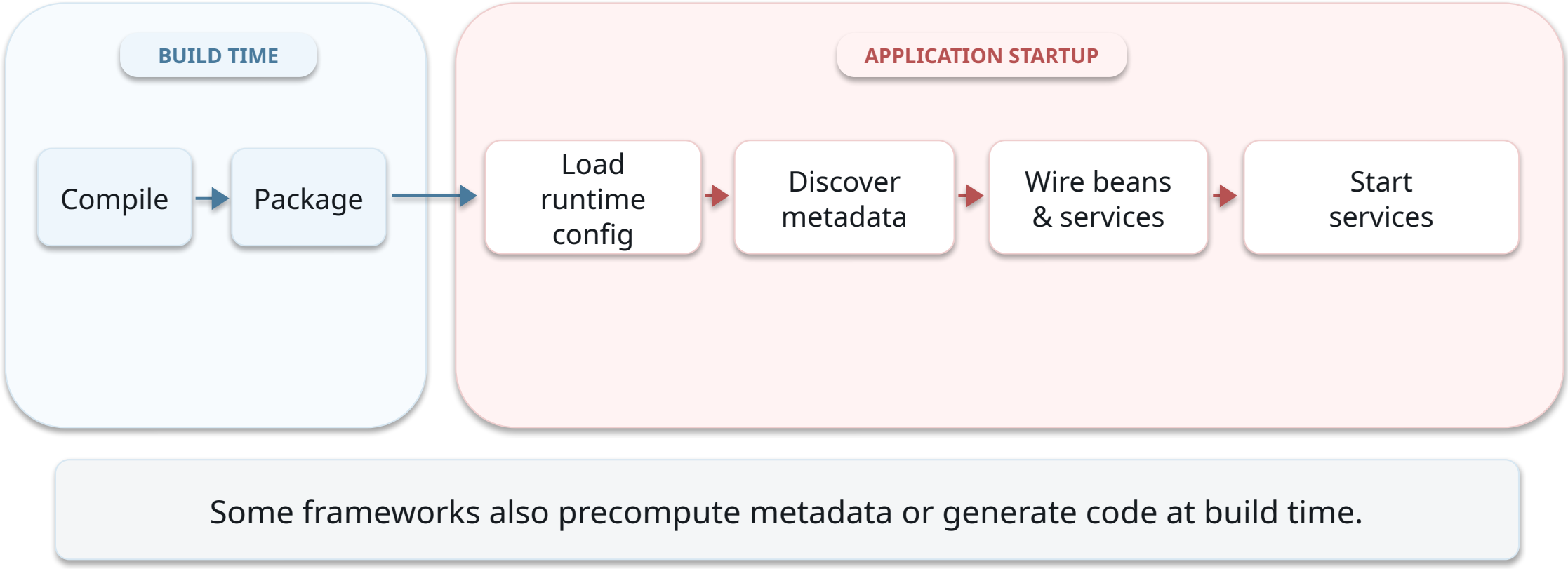
Platform integration

Support native images, dev mode and extension conventions.

Move the right work to build time - not everything.

Runtime-heavy framework bootstrap

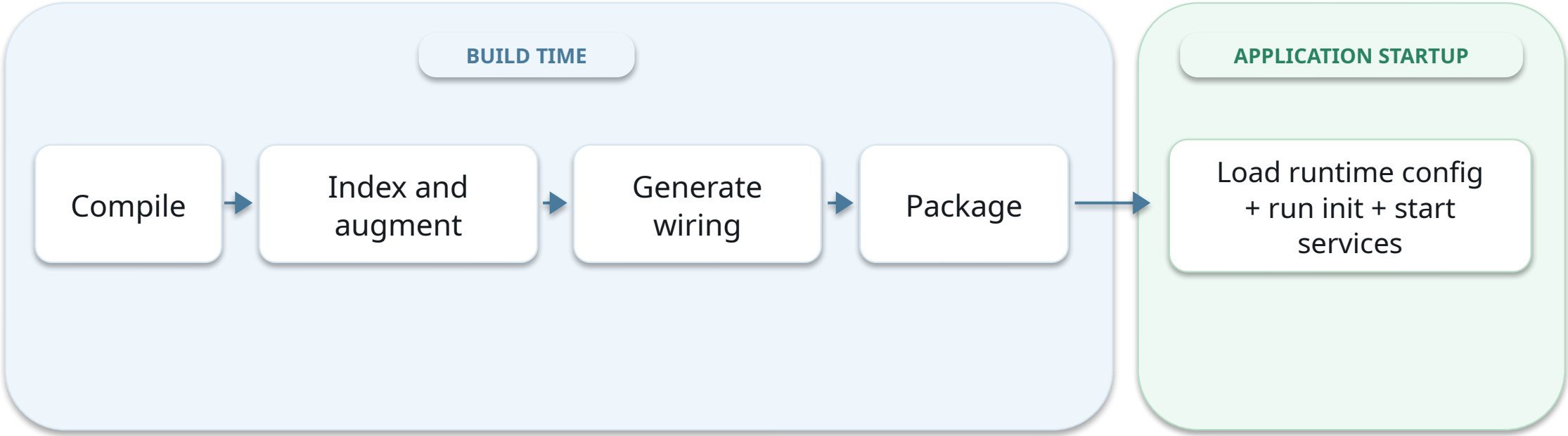
An illustrative application model - behavior varies by framework and configuration.



This comparison illustrates a runtime-heavy design, not every Java framework.

Quarkus application bootstrap

Quarkus performs augmentation at build time while retaining necessary startup work.



JVM: recorded static init then runtime init at startup.
Native: static init can run during native image build.

Less runtime discovery does not mean zero runtime initialization.

Quarkus shifts work earlier

Build steps perform augmentation; prepared runtime code does less discovery.



Analyze classes, produce build items, generate wiring

Recorded initialization safe for static phase

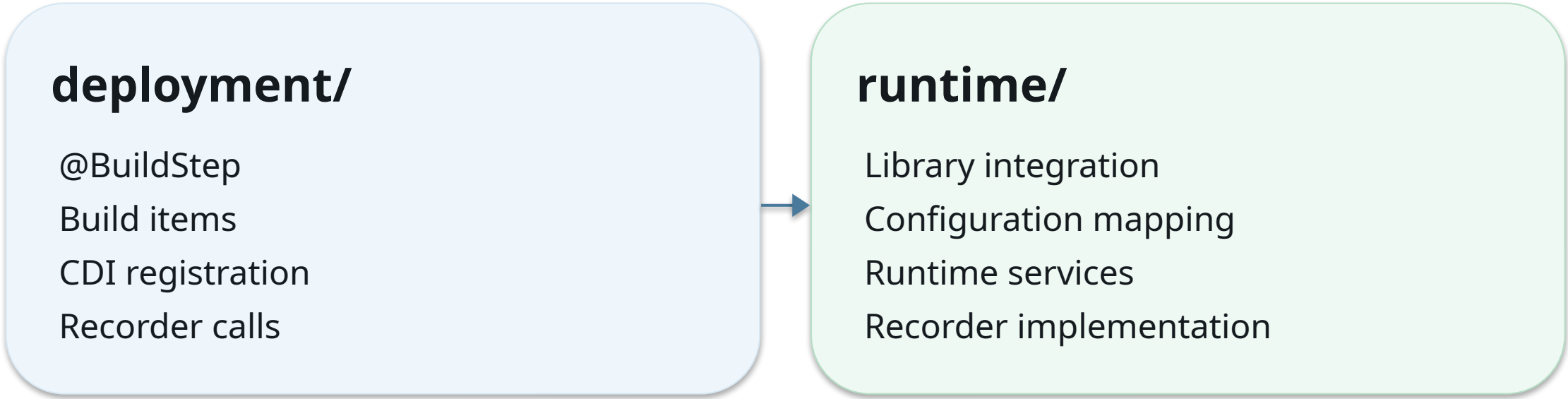
Initialize services needing runtime state

Build step executes now; a recorded action can execute in a later phase.

Build time, static init and runtime init are not synonyms.

An extension has two halves

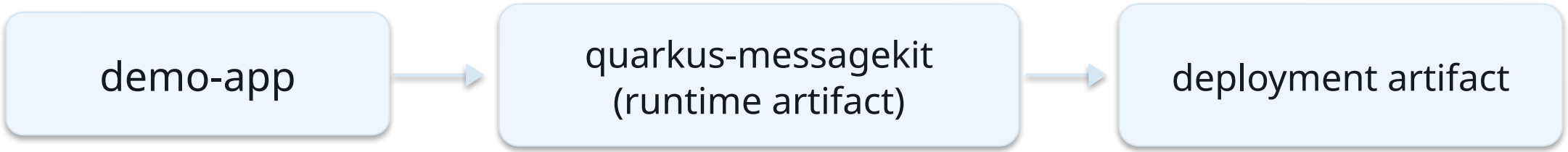
The split lets build-time code prepare a runtime application.



Deployment code participates in augmentation; runtime code remains with the app.

How Quarkus finds the deployment side

The application depends on the runtime artifact, not on deployment directly.




```
META-INF/quarkus-extension.yaml

quarkus-extension-maven-plugin
generates the bootstrap descriptor
```

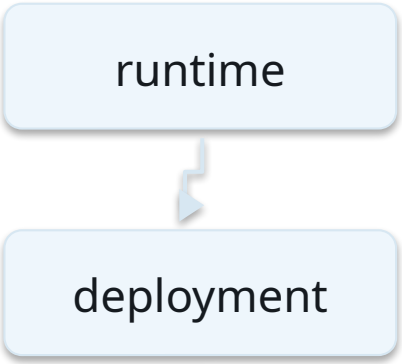
The runtime extension carries the metadata that connects both halves.

The extension skeleton

Generated layout makes the runtime / deployment contract explicit.

DEMO 01 **Inspect Maven modules and the first build step**

```
quarkus-messagekit/
  pom.xml
  runtime/
    pom.xml
    src/main/resources/META-INF/quarkus-extension.yaml
  deployment/
    pom.xml
    src/main/java/.../MessageKitProcessor.java
```

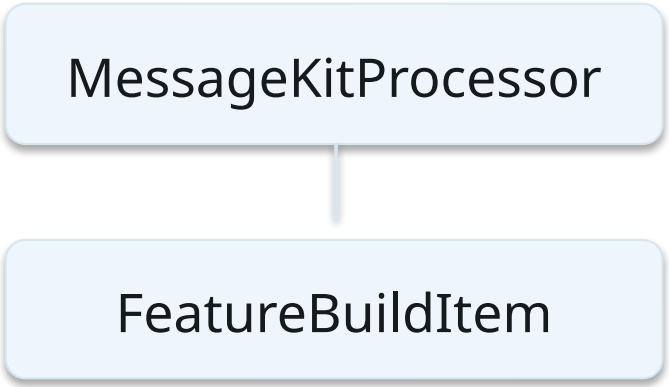


The empty skeleton already participates in the Quarkus build.

A build step is an augmentation action

Quarkus discovers @BuildStep methods and orchestrates them using build items.

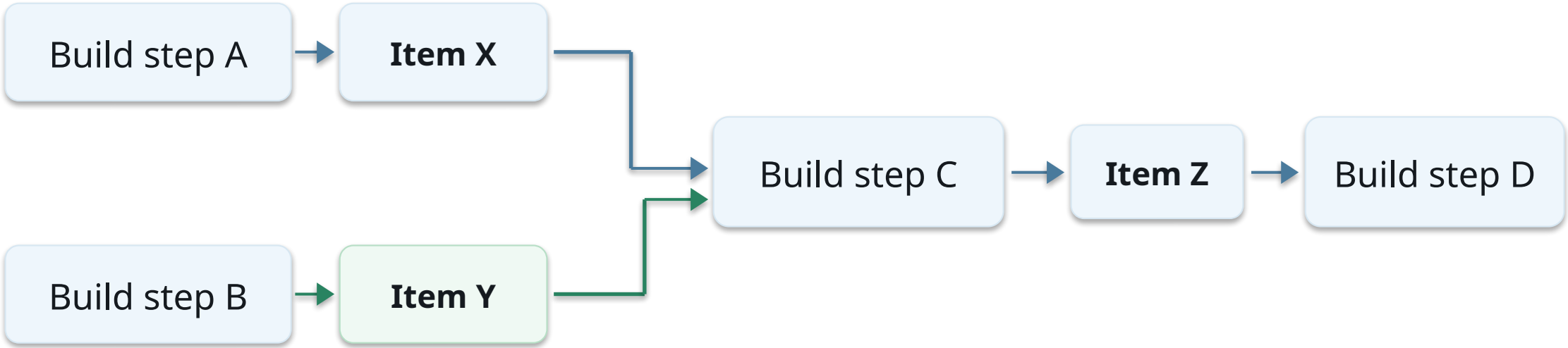
```
@BuildStep
FeatureBuildItem feature() {
    return new
FeatureBuildItem("messagekit");
}
```



Build items are the communication mechanism between build steps.

Quarkus build flow: a dependency graph

Build items connect producers and consumers; Quarkus derives the required build steps.



Illustrative only: A and B are independent. C consumes X and Y. D consumes Z.

No global list of callbacks: build item dependencies determine the graph.

Let Quarkus discover the producer

The runtime producer exists, but augmentation must register it for CDI.

DEMO 02

Register CDI integration with a build item

```
@BuildStep
AdditionalBeanBuildItem registerFormatterProducer() {
    return AdditionalBeanBuildItem.builder()
        .addBeanClass(MessageFormatterProducer.class)
        .setUnremovable()
        .build();
}
```

CDI bean registered

The consuming application no longer owns integration boilerplate.

What changes for the app developer?

The extension makes the integration reusable without rewriting the consuming code.

BEFORE: application-owned

MessageFormatterProducer.java
Manual bean setup
Library configuration in each app



AFTER: extension-owned

Runtime extension dependency
@Inject MessageFormatter
Quarkus configuration properties

The win is not fewer lines of code - it is consistent integration.

Configuration is a public API

Expose settings once; consuming applications use standard Quarkus properties.

```
@ConfigRoot(phase = ConfigPhase.RUN_TIME)
@ConfigMapping(prefix = "quarkus.messagekit")
public interface MessageKitConfig {
    @WithDefault("en") String language();
    Optional<String> prefix();
    @WithDefault("false") boolean uppercase();
}
```

Public contract

Names

Defaults

Validation

Config phase

Configuration property names are part of your extension API.

Change behavior without changing code

A runtime config mapping lets the app select language, prefix and case.

DEMO 03 **Run the same endpoint with Croatian settings**

```
quarkus.messagekit.language=hr
quarkus.messagekit.prefix=Poruka
quarkus.messagekit.uppercase=true
```

GET /hello/Ana → **PORUKA POZDRAV ANA**

Same Java code. Different runtime behavior.

Static init and runtime init are different

Native images make phase selection particularly important.

AUGMENTATION	`@BuildStep runs`	Analyze and prepare information
STATIC_INIT	`@Record(STATIC_INIT)`	Only initialization safe for the static phase
RUNTIME_INIT	`@Record(RUNTIME_INIT)`	Runtime-dependent state and services

A recorder call is declared at build time; its work runs later.

The recorder owns the runtime creation logic

Keep the recorder in runtime and call it from a recorded build step.

```
@Recorder
public class MessageKitRecorder {
    public RuntimeValue<MessageCatalog> createCatalog(
            String en, String es, String hr) {
        return new RuntimeValue<>(new MessageCatalog(
            Map.of("en", en, "es", es, "hr", hr)));
    }
}
```

The build chooses the inputs; the recorder constructs the runtime value.

Record initialization; expose a synthetic bean

MessageCatalog is created at RUNTIME_INIT and made injectable.

DEMO 04

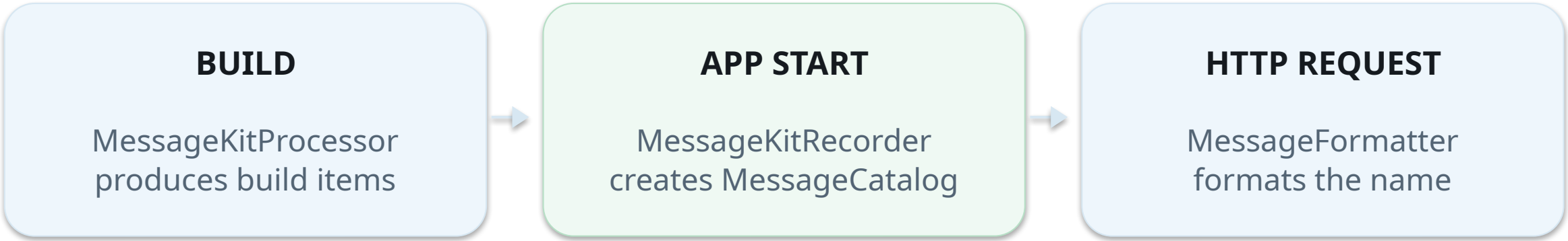
Connect @Record to SyntheticBeanBuildItem

```
@BuildStep
@Record(RUNTIME_INIT)
SyntheticBeanBuildItem createMessageCatalog(
        MessageKitRecorder recorder) {
    return SyntheticBeanBuildItem.configure(MessageCatalog.class)
        .scope(Singleton.class)
        .setRuntimeInit()
        .runtimeValue(recorder.createCatalog(
            "Hello", "Hola", "Pozdrav"))
        .done();
}
```

The deployment module describes the bean; runtime supplies its value.

What actually runs when?

The most important distinction in the entire extension lifecycle.



The timing of initialization is part of correctness.

Test the extension, not just the library

Extension tests verify the consuming application context and augmentation.

Plain JUnit

Does MessageFormatter behave correctly as a Java class?

QuarkusExtensionTest

Are extension-created beans and config available to applications?

Demo app tests

Does the REST-facing behavior work end to end?

Each test layer answers a different question.

One test should tell a complete story

Readable JUnit DisplayNames and Croatian data make the contract visible.

DEMO 05

Verify Croatian configuration and injectable beans

```
@RegisterExtension
static final QuarkusExtensionTest app =
    new QuarkusExtensionTest().withEmptyApplication()
        .overrideRuntimeConfigKey("quarkus.messagekit.language", "hr")
        .overrideRuntimeConfigKey("quarkus.messagekit.prefix", "Poruka")
        .overrideRuntimeConfigKey("quarkus.messagekit.uppercase", "true");

@Inject MessageFormatter formatter;

@Test
@DisplayName("Applies Croatian language, prefix, and uppercase formatting")
void croatianRuntimeConfigurationChangesTheMessage() {
    assertEquals("PORUKA POZDRAV ANA", formatter.format("Ana"));
}
```

The happy path is only the beginning

Extensions must behave correctly across build modes and deployments.

Bean discovery

Know why a bean is indexed, registered or removed as unused.

Init phase

Do not freeze runtime-dependent values into build-time state.

Native image

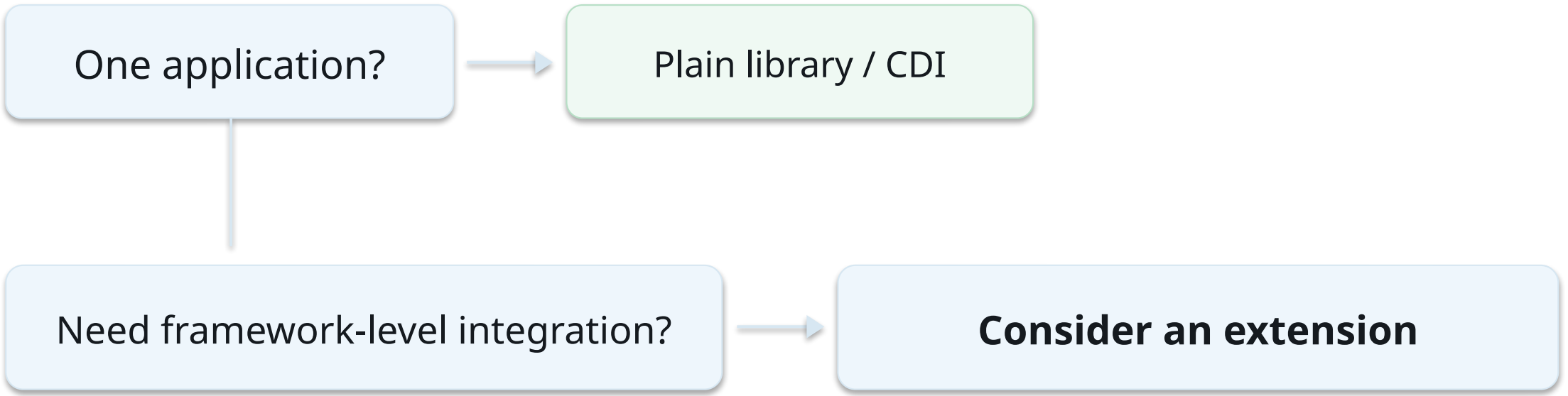
Check reflection, resources and runtime initialization deliberately.

Dev mode, dependency alignment, and compatibility tests complete the picture.

An extension is a long-lived integration contract.

When should you build an extension?

Ask about architectural needs before choosing the integration mechanism.



A Quarkus extension earns its complexity by removing more complexity elsewhere.

Three ideas to remember

These matter more than memorizing any extension API.

Integrate intentionally

Start with the smallest valid solution; add an extension when justified.

Respect the phases

Build steps, static init and runtime init have distinct roles.

Test as a consumer

Verify beans, config, dev experience and native-related behavior.

Build less runtime machinery by making better build-time decisions.

Thank you

Questions?

GitHub Repository:
https://github.com/petervelychko/quarkus-extensions-demo

Building Quarkus Extensions: A Practical Guide for Java Developers